Visão Computacional TA02 - Calibração

Marcelo Gyovani Pereira Vinícius Evair da Silva

Repositório: `https://github.com/USUARIO/REPOSITORIO`

1 Introdução

A calibração de uma câmera estima os parâmetros que relacionam pontos de uma cena às suas posições na imagem. A matriz intrínseca reúne as distâncias focais em pixels e o ponto principal; os coeficientes de distorção representam desvios em relação ao modelo de projeção ideal. Esses parâmetros permitem corrigir deformações geométricas e projetar pontos tridimensionais em coordenadas de imagem, conforme os procedimentos do OpenCV [1, 2].

Neste trabalho, foram calibradas duas câmeras de um smartphone. Os parâmetros obtidos foram aplicados à correção de fotografias de teste e à projeção de pontos de um tabuleiro. O principal experimento quantitativo utiliza novas fotografias, não incluídas na calibração, e compara posições observadas e previstas de quatro cantos reservados da estimação da pose em cada imagem.

2 Metodologia

Foram utilizadas as câmeras principal, no modo 1x (`cam1`), e ultrawide, no modo 0,5x (`cam05`), de um Samsung Galaxy A56. A coleta inicial reuniu 22 fotografias do tabuleiro por câmera, com resolução de 2992×2992 pixels, variando posição, distância e inclinação do padrão. O tabuleiro possui 5×5 quadrados de lado 3,1 cm e 4×4 cantos internos. Os dados foram organizados em diretórios independentes para cada câmera. Para avaliar a projeção, foram obtidas outras três fotografias do tabuleiro por câmera, armazenadas em `project_test/` e mantidas fora da calibração.

A implementação utiliza Python, NumPy e OpenCV. Em `calibrate.py`, os cantos são detectados com `cv2.findChessboardCorners()`, com `cv2.findChessboardCornersSB()` como alternativa em caso de falha, e refinados por `cv2.cornerSubPix()`. O refinamento utiliza janela de 11×11 pixels, limite de 30 iterações e tolerância de 0,001 pixel. A Figura 1 apresenta uma detecção existente no repositório.

A calibração utiliza `cv2.calibrateCamera()`, que recebe as coordenadas do padrão e as observações 2D. Em seguida, `undistort.py` aplica `cv2.undistort()` às fotografias, enquanto `project.py` estima a pose com `cv2.solvePnP()` e calcula projeções com `cv2.projectPoints()`. Todos os cálculos utilizam a resolução original; os detalhes ampliados nas figuras servem apenas à apresentação.

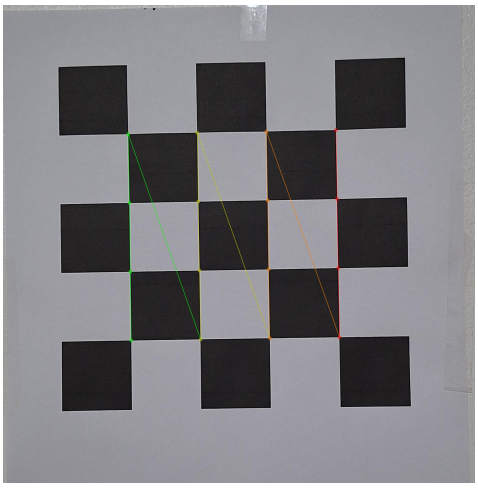


Figura 1: Detalhe da fotografia `001.jpg` da câmera 1x, com os 16 cantos internos detectados e conectados pelo OpenCV.

3 Calibração das câmeras

A calibração final utilizou 22 fotografias da câmera 1x e 20 da câmera 0,5x. Conforme o histórico experimental informado, a primeira calibração da câmera 0,5x empregou as 22 fotografias; as imagens 019 e 022 apresentaram os maiores erros individuais e foram retiradas manualmente. Após essa seleção, o RMS passou de aproximadamente 1,686 para 1,463 pixel. Os arquivos atuais registram o ajuste final com 20 imagens, sem novos descartes na execução.

Os arquivos `calibration.npz` e `log.txt`, nos diretórios de cada câmera, registram os mesmos parâmetros. As matrizes intrínsecas, em pixels, são

$$K_{1x} = \begin{bmatrix} 2715{,}936 & 0 & 1484{,}581 \\ 0 & 2706{,}305 & 1520{,}002 \\ 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

$$K_{0{,}5x} = \begin{bmatrix} 1600{,}726 & 0 & 1586{,}526 \\ 0 & 1596{,}602 & 1494{,}484 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

As menores distâncias focais em pixels da câmera 0,5x são compatíveis com seu enquadramento mais amplo; não representam medidas físicas em milímetros.

Tabela 1: Resultados do ajuste final. Os erros são expressos em pixels.

Câmera	Imagens	RMS	Erro médio euclidiano
Principal (1x)	22	0,910	0,667
Ultra-angular (0,5x)	20	1,463	1,168

A Tabela 2 apresenta os coeficientes adimensionais na convenção do OpenCV: k_1, k_2 e k_3 são radiais; p_1 e p_2 são tangenciais. A notação científica mantém visíveis os coeficientes de menor magnitude.

Tabela 2: Coeficientes de distorção das duas câmeras.

Coeficiente	Principal (1x)	Ultra-angular (0,5x)
k_1	$1{,}041 \times 10^{-1}$	$-1{,}245 \times 10^{-2}$
k_2	$-3{,}006 \times 10^{-1}$	$3{,}508 \times 10^{-2}$
k_3	$9{,}202 \times 10^{-5}$	$1{,}756 \times 10^{-2}$
p_1	$2{,}460 \times 10^{-3}$	$8{,}152 \times 10^{-4}$
p_2	$1{,}630 \times 10^{-3}$	$1{,}142 \times 10^{-2}$

O RMS é a raiz da média das distâncias de reprojeção ao quadrado. O erro médio é a média das distâncias euclidianas entre cantos detectados e reprojetados. Com 16 pontos por imagem, a média das médias por imagem coincide com a global.

A câmera principal apresentou menor erro nas duas medidas da Tabela 1. Campo de visão, tamanho do padrão na imagem, localização dos cantos e adequação do modelo são possíveis fatores para essa diferença, mas não foram isolados experimentalmente. Além disso, a redução do RMS após retirar imagens da câmera 0,5x mede o ajuste ao conjunto selecionado.

4 Correção da distorção

Os parâmetros de cada câmera foram reutilizados em `undistort.py` para corrigir a distorção geométrica com `cv2.undistort()`. A função `cv2.getOptimalNewCameraMatrix()` determina uma nova matriz intrínseca para a mesma resolução, com duas escolhas de α [1]. Para $\alpha = 0$, prioriza-se uma região válida, evitando bordas pretas e reduzindo o campo de visão. Para $\alpha = 1$, preserva-se mais do campo original, com possíveis regiões sem pixels correspondentes, representadas em preto. O programa não recorta a região de interesse retornada nem redimensiona os arquivos corrigidos.

Foram utilizadas fotografias de teste sem tabuleiro, separadas das imagens de calibração. As Figuras 2 e 3 reproduzem comparações já geradas para `fogao1.jpg`. As bordas da porta e os rejuntes dos azulejos, próximos aos limites das fotografias, permitem inspecionar visualmente as alterações. Cada câmera observa a cena com enquadramento próprio; a comparação relevante ocorre entre as versões da mesma fotografia.



Figura 2: Câmera principal (1x), fotografia de teste `fogao1.jpg`. Da esquerda para a direita: original, corrigida com $\alpha = 1$ e corrigida com $\alpha = 0$. A faixa preta curva na versão central corresponde a regiões sem amostras da imagem original.



Figura 3: Câmera ultrawide (0,5x), fotografia de teste `fogao1.jpg`. A ordem é original, $\alpha = 1$ e $\alpha = 0$. Observam-se o remapeamento da cena, as bordas sem correspondência e a mudança de enquadramento.

As diferenças locais podem ser discretas, embora a alteração de enquadramento seja visível. Não foi medida a retidão das linhas ou outra métrica geométrica nas imagens de teste; portanto, essas figuras não demonstram uma melhoria quantitativa da geometria. A correção também não elimina efeitos de perspectiva, como a convergência de linhas associada à orientação da câmera em relação à cena. Bordas pretas, por sua vez, decorrem do domínio do remapeamento e não representam regiões adicionais capturadas pela câmera.

5 Projeção de pontos 3D em imagens 2D

5.1 Estimação da pose e separação dos pontos

O experimento de `project.py` utiliza os parâmetros já salvos e três fotografias inéditas por câmera, de `project_test/`, sem recalibrar as câmeras. O mesmo procedimento é aplicado às fotografias da calibração para comparação. Os 16 cantos do tabuleiro têm coordenadas conhecidas $\mathbf{P}_{r,c} = (3{,}1c,\ 3{,}1r,\ 0)$ cm, com $r, c \in \{0, 1, 2, 3\}$. O índice é $4r + c$, com a coluna variando primeiro, conforme a ordenação usada na calibração. A origem está no canto de índice zero. Essas coordenadas pertencem ao referencial local do tabuleiro, e não a um sistema global.

Após a detecção e o refinamento dos cantos, 12 correspondências entre pontos 3D e posições 2D observadas estimam a rotação e a translação do tabuleiro por uma única chamada a `cv2.solvePnP()`, no modo `SOLVEPNP_ITERATIVE`. A pose transforma o referencial do tabuleiro no referencial da câmera [2]. A seleção é determinística: os índices 0, 3, 12 e 15, situados nos quatro extremos da grade interna, ficam reservados; os outros 12 se distribuem pelas quatro linhas e colunas. Os pontos reservados não entram no `solvePnP()` nem em um refinamento posterior da pose.

Com a pose estimada, `cv2.projectPoints()` calcula as coordenadas previstas dos quatro pontos reservados, usando a matriz intrínseca, os coeficientes de distorção e os vetores de rotação e translação. A comparação é feita no domínio da fotografia original: não são usadas as imagens corrigidas por `undistort.py`, pois a projeção já incorpora a distorção. O procedimento é repetido por fotografia, com 12 pontos de pose e quatro de validação.

5.2 Métricas e resultados

Se (u_i, v_i) é a posição observada e $(\hat{u}_i, \hat{v}_i)$ a posição prevista, o erro individual, em pixels, é

$$e_i = \sqrt{(u_i - \hat{u}_i)^2 + (v_i - \hat{v}_i)^2}. \tag{3}$$

Para os N pontos de validação de uma câmera, calculam-se

$$\bar{e} = \frac{1}{N}\sum_{i=1}^{N} e_i, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}, \qquad e_{\max} = \max_i e_i. \tag{4}$$

A média descreve o desvio típico, o RMSE atribui maior peso aos erros elevados e o máximo registra o maior desvio individual. Todas as métricas globais utilizam os erros ponto a ponto; não se faz a média dos RMSEs por imagem.

As três fotografias inéditas de cada câmera foram processadas sem falhas de detecção ou estimação da pose, totalizando 12 pontos de validação por câmera. Os resultados estão em `projections/tests/summary.txt` e `points.csv`. A Tabela 3 também apresenta a referência obtida nas 22 fotografias de calibração da câmera 1x e nas 20 da câmera 0,5x, com 88 e 80 pontos reservados, respectivamente. As métricas de cada conjunto foram calculadas separadamente, sem excluir erros elevados.

Tabela 3: Erros de projeção em pixels: fotografias da calibração e teste inédito.

Câmera	Conjunto	Imagens	Média	RMSE	Máximo
1x	Calibração	22	1,121	1,575	5,121
1x	Teste inédito	3	2,839	4,061	9,162
0,5x	Calibração	20	2,112	2,563	6,119
0,5x	Teste inédito	3	2,418	3,060	6,821

As Figuras 4 e 5 apresentam fotografias inéditas das duas câmeras, com detalhe da região do tabuleiro. Os marcadores são maiores apenas para facilitar a leitura; as distâncias da Equação (3) foram calculadas com coordenadas de subpixel. Como os desvios são pequenos na escala das figuras, observações e projeções podem aparecer quase sobrepostas.

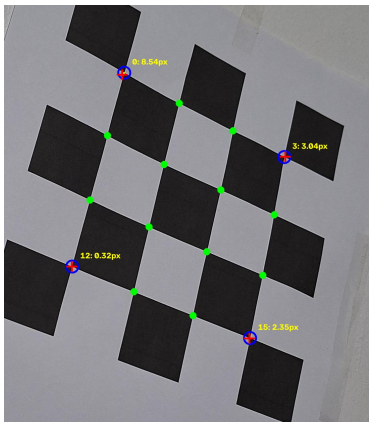


Figura 4: Câmera 1x, fotografia inédita `001.jpg`. Verde: pose; azul: observações reservadas; vermelho: projeções. Amarelo: conexões e rótulos de erro. Média nesta imagem: 3,562 px.

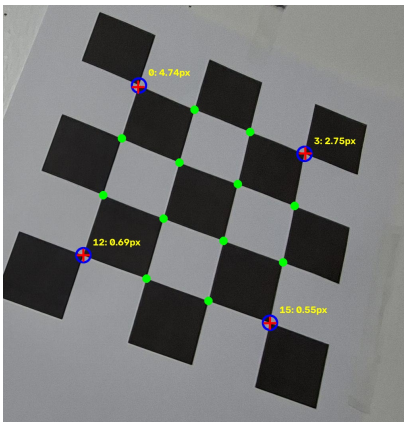


Figura 5: Câmera 0,5x, fotografia inédita `002.jpg`, com a convenção da Figura 4. Média nesta imagem: 2,183 px. As duas figuras foram enquadradas somente no documento.

Nas fotografias inéditas, o erro médio aumentou 153,19% na câmera 1x e 14,50% na câmera 0,5x; o RMSE aumentou 157,75% e 19,41%, respectivamente. A câmera 0,5x apresentou menores erros nesse novo conjunto, embora a câmera principal tivesse menores erros nas fotografias da calibração. A escala e a posição do tabuleiro, sua inclinação e resíduos do modelo podem influenciar a diferença; esses fatores não foram isolados.

As novas fotografias não participaram da calibração intrínseca, e os quatro cantos reservados também não participaram da estimação da pose. A avaliação verifica, portanto, a projeção em imagens inéditas, mas a pose de cada imagem continua sendo estimada a partir de 12 correspondências conhecidas. Com apenas três imagens por câmera, os resultados descrevem esta amostra e não estabelecem uma superioridade geral de uma das câmeras.

6 Conclusão

A calibração foi realizada para as câmeras principal e ultrawide do smartphone, obtendo-se matrizes intrínsecas e coeficientes de distorção que foram reutilizados na correção de fotografias. Na validação da projeção 3D–2D em três fotografias inéditas por câmera, os erros médios foram aproximadamente 2,84 pixels para a câmera 1x e 2,42 pixels para a câmera 0,5x. Os erros aumentaram em relação às fotografias da calibração, sobretudo na câmera principal. Esta manteve os menores erros de calibração, enquanto a câmera 0,5x apresentou menores erros de projeção na nova amostra.

As conclusões são limitadas pelo tamanho e pela seleção dos conjuntos, pelo uso de um padrão plano e pelo número reduzido de fotografias inéditas. A pose depende de correspondências 2D conhecidas em cada imagem, e a correção das fotografias de teste foi examinada apenas visualmente. Um conjunto de teste mais amplo, com condições de captura controladas e medidas de geometria, permitiria avaliar melhor a generalização dos parâmetros.

Referências

[1] OPENCV. *Camera Calibration*. OpenCV-Python Tutorials. Disponível em: `https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html`. Acesso em: 17 out. 2026.

[2] OPENCV. *Camera Calibration and 3D Reconstruction*. Documentação do módulo calib3d. Disponível em: `https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html`. Acesso em: 17 out. 2026.